UCS2611 INTERNET PROGRAMMING LAB

MINI PROJECT REPORT

TAX CALCULATOR AND FILING ASSISTANT

LAKSHMAN VIJAY VI 3122235001072

MADUSSREE RAVI 3122235001074

COMPUTER SCIENCE AND ENGINEERING

SSN COLLEGE OF ENGINEERING
KALAVAKKAM 603110

Introduction

The Tax Calculator Application is a full-stack web-based system designed to compute tax liabilities based on user-provided financial data with an Indian context. It enables users to input income and deductions, perform tax calculations, visualize results, and store historical records, whilst comparing Old and New Tax regimes. The comparison enables the user to figure out the most suitable regime with proper reasoning and also simulate what-if scenarios depending on salary and allowance remittance. The Application also includes an AI assistant to guide the user with user context and given context window

System Architecture Overview

The application follows a client-server architecture using the MERN stack with enhancements for performance and deployment.

Frontend

The frontend is a Single Page Application built using React and Vite. It is responsible for user interaction, collecting financial inputs, managing authentication state, and displaying results through dynamic UI components and charts.

Backend

The backend is developed using Node.js and Express. It exposes RESTful APIs, processes business logic, validates user input, performs authentication using JSON Web Tokens, and communicates with the database.

Database

The database layer uses MongoDB, a NoSQL document database, allowing flexible schema design. Mongoose is used as the Object Data Modeling library to structure and manage application data.

Data Models

1) User

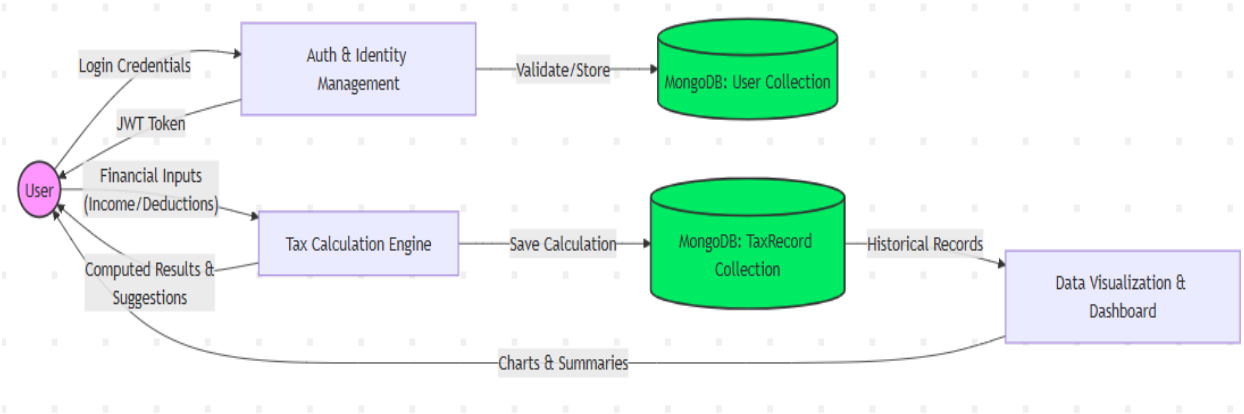
name	email	password	role	created_at

2) Tax Records

user	year	income	deductions	calculation

regimeComparison	recommendedRegime	regimeSavings	suggestions

Data Flow Diagram



Tech Stack

Layer	Technology	Key Function
Frontend	**React + Vite**	**SPA development with fast HMR (Hot Module Replacement).**
Backend	**Node.js + Express**	**RESTful API management and business logic.**
Database	**MongoDB + Mongoose**	**NoSQL storage with Object Data Modeling (ODM).**
Auth	**JWT**	**Secure, stateless authentication and password hashing.**

Core System Modules

Authentication & Authorization

- **Security:** Passwords are salted and hashed via **Bcrypt**.
- **Access Control:** Uses **JSON Web Tokens (JWT)** for stateless sessions.
- **Middleware:** A dedicated authorization layer intercepts requests to protect sensitive endpoints.

Tax Calculation Engine

- **Aggregation:** Summing income from salary, business, and freelance sources.
- **Deductions:** Applying caps to categories like medical expenses and investments.
- **Progressive Logic:** Applying multi-tier tax brackets to calculate final liability.
- **Optimization:** Providing AI-style suggestions for unused deduction limits.

Dashboard & Assessment

- **Data Entry:** Dynamic forms for income and expense tracking.
- **Visualization:** Integrated **Chart.js** to provide visual breakdowns of income vs. tax.
- **Persistence:** Historical records are linked via User IDs for year-over-year comparisons.

Tax Engine Computation Logic

The taxEngine.js module handles all tax calculations on the server using Indian Income Tax rules. It computes both old and new regimes and returns a full comparison along with a recommendation.

Income Calculation

- Total income is calculated by combining salary, business, freelance, and other sources.
- HRA exemption is computed using standard rules based on rent, salary, and metro/non-metro limits.

Old Regime

- Standard deduction of ₹50,000 is applied.
- Deductions include:
    - 80C (up to ₹1.5L)
    - 80D (₹25K–₹50K depending on age)
    - NPS 80CCD(1B) (₹50K)
- Tax is calculated using traditional slabs (0% to 30%).
- 87A rebate applies if income is up to ₹5L.

New Regime

- Standard deduction of ₹75,000 is applied.
- Most deductions (80C, 80D, HRA) are not allowed.
- Uses revised slab structure (0% to 30%).
- 87A rebate applies up to ₹7L.
- Includes marginal relief near ₹7L.
- Surcharge is capped at 25%, with 4% cess applied.

Recommendation

- The system compares both regimes and picks the one with lower tax.
- Shows exact savings between the two.
- Suggests ways to save more tax based on unused deductions.

AI and Interactive Features

AI Chat Assistant

- A chat interface that answers tax-related questions using the user's data.
- Automatically generates a short summary of the latest tax record.
- Supports common queries like reducing tax or choosing the right regime.
- Uses a lightweight AI model with controlled response length.

Tax Simulator

- A simple interactive tool with sliders for salary, investments, and HRA.
- Updates tax values instantly without calling the backend.
- Displays results visually using charts.
- Highlights the better tax regime in real time.

Document Parser

- Users can upload PDF or image documents like salary slips.
- The system extracts key financial details using AI.
- Returns structured data along with a confidence level.
- Allows one-click autofill into the calculator form.

Regime Comparison View

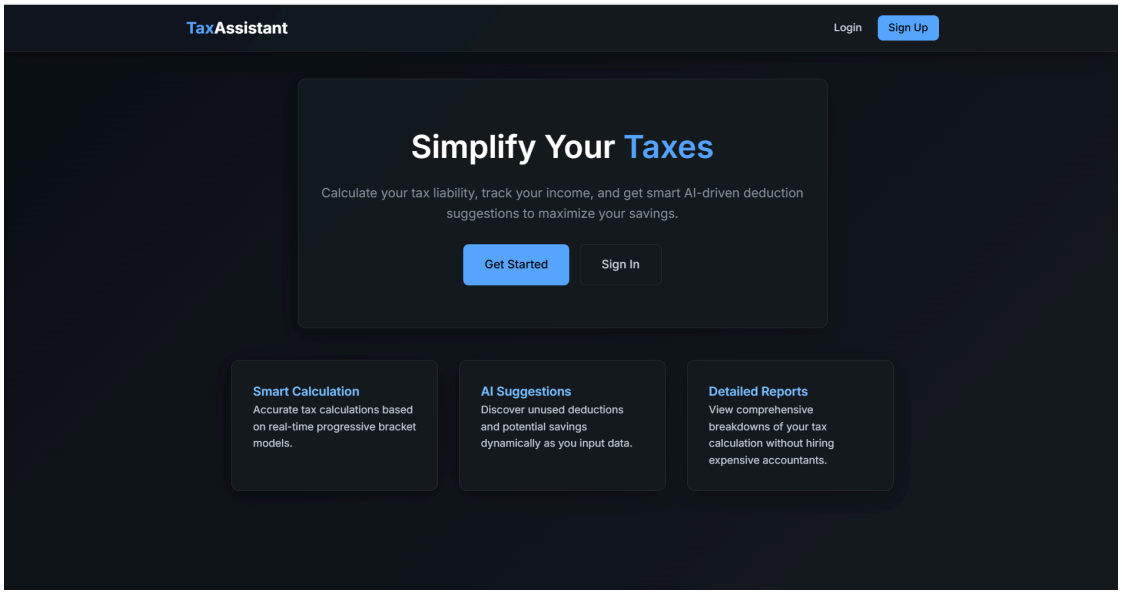
- Shows old vs new regime side by side.
- Includes full breakdown: taxable income, tax, rebate, cess, and total.
- Clearly highlights which regime is better and how much is saved.
- Uses precomputed data, so no extra API calls are needed.

Communication Endpoints

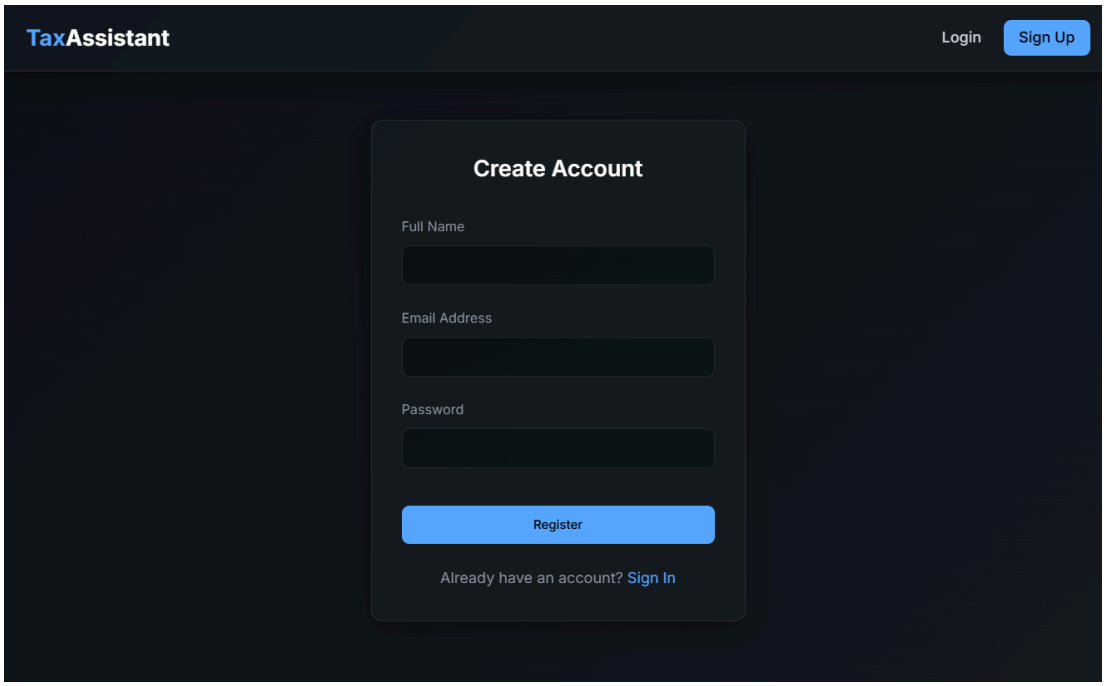
Endpoint	Method	Purpose
/api/auth/login	POST	Authenticate User
/api/tax/calculate	POST	Process Tax
/api/tax/history	GET	Retrieve Records
/api/auth/me	GET	Fetch currently authenticated user's profile (password excluded)
/api/calculations	POST	Computes old & new regime, 87A and regime recommendation
/api/ai/chat	POST	Sends user message with context
/api/upload/parse	POST	Accept PDF/JPG/PNG of salary slip or Form 16, extract structured tax fields

Screenshots of the Project

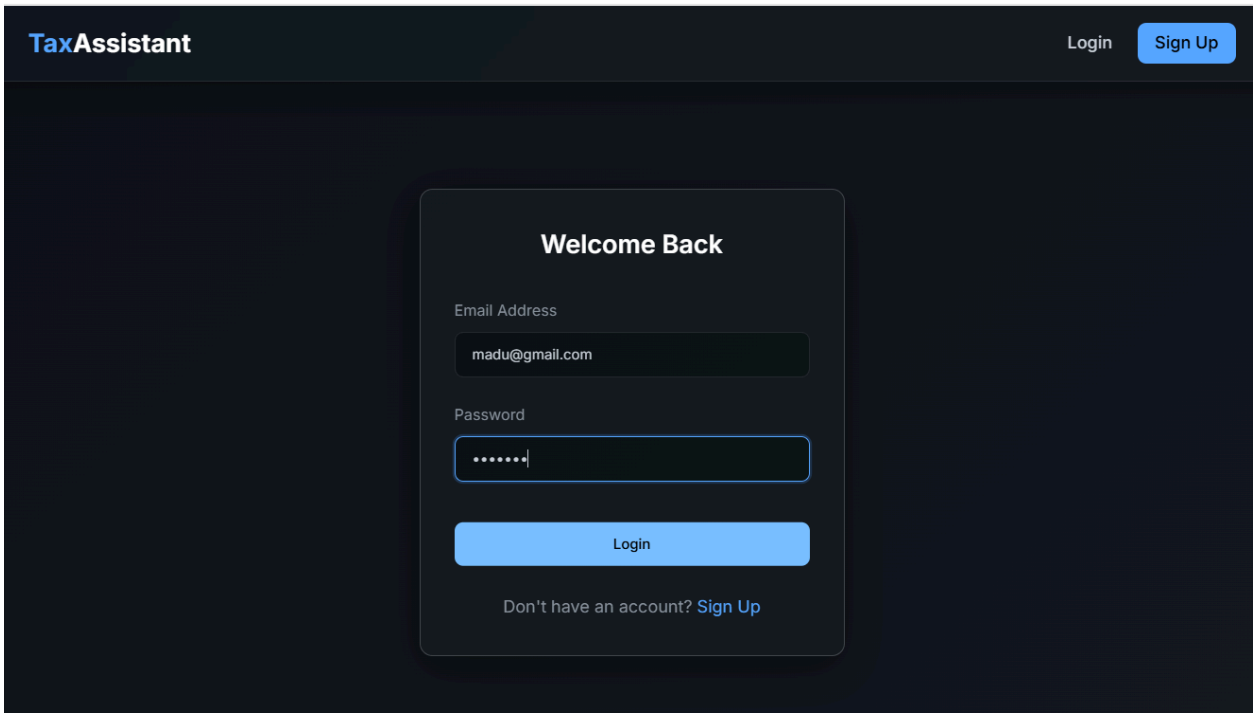
1) Landing Page



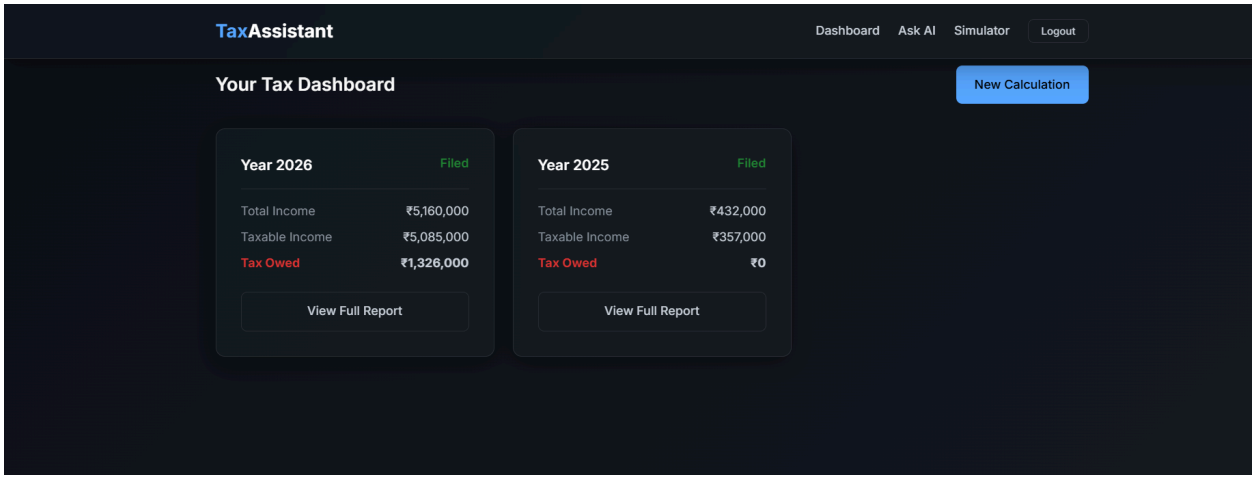
2) Sign Up Page



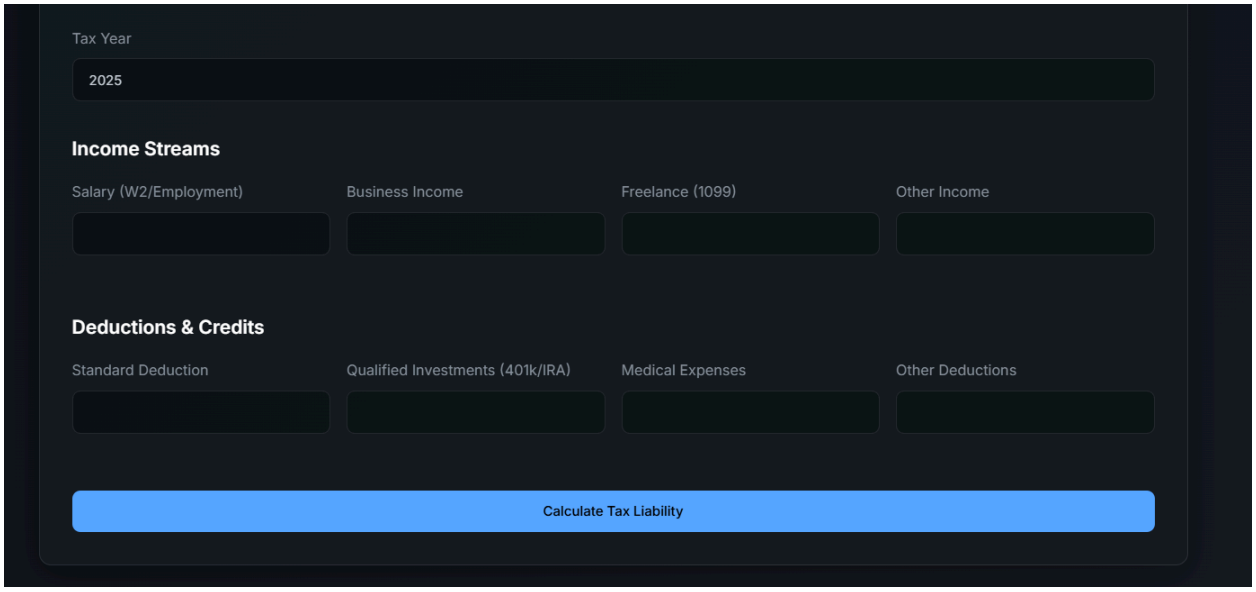
3) Register Page



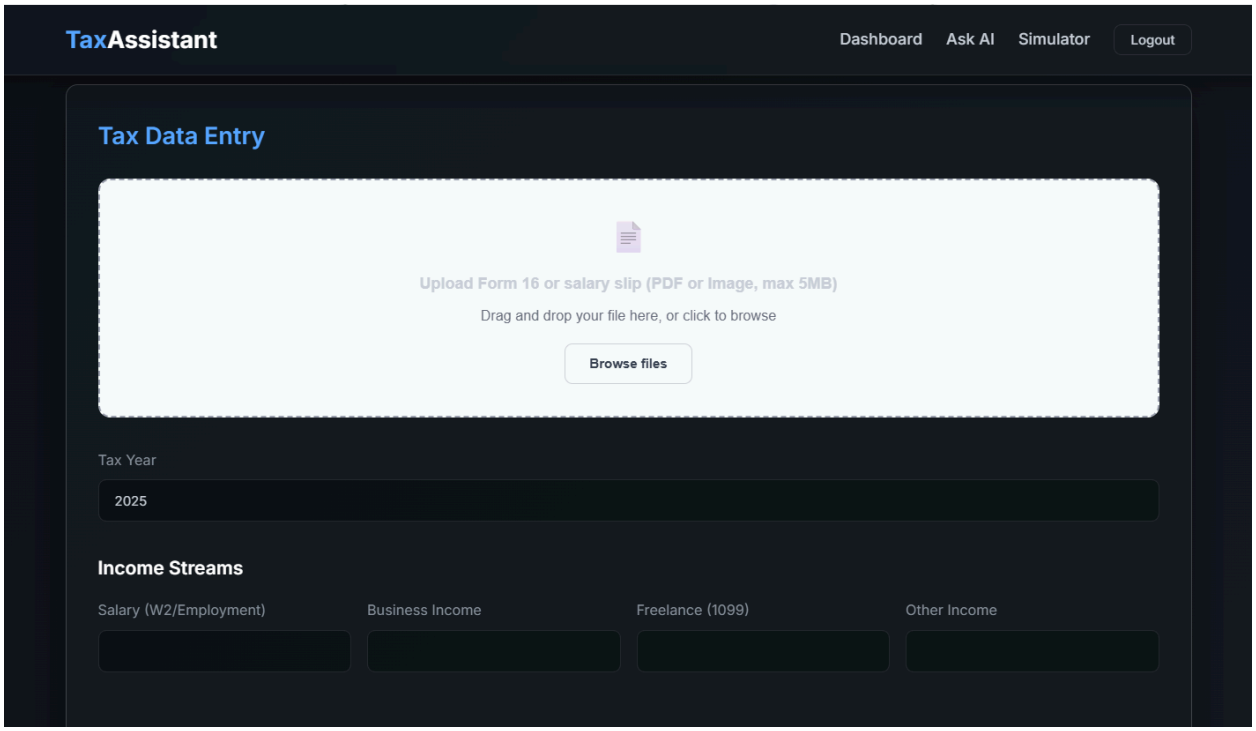
4) Dashboard



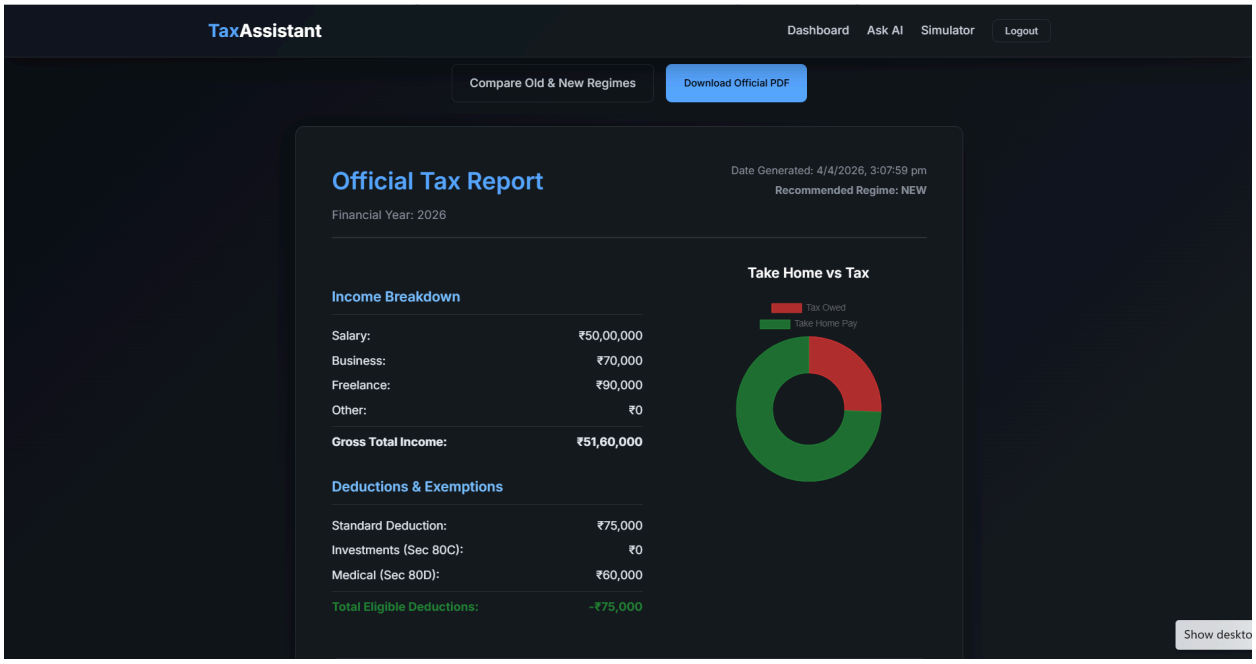
5) Adding a new calculation



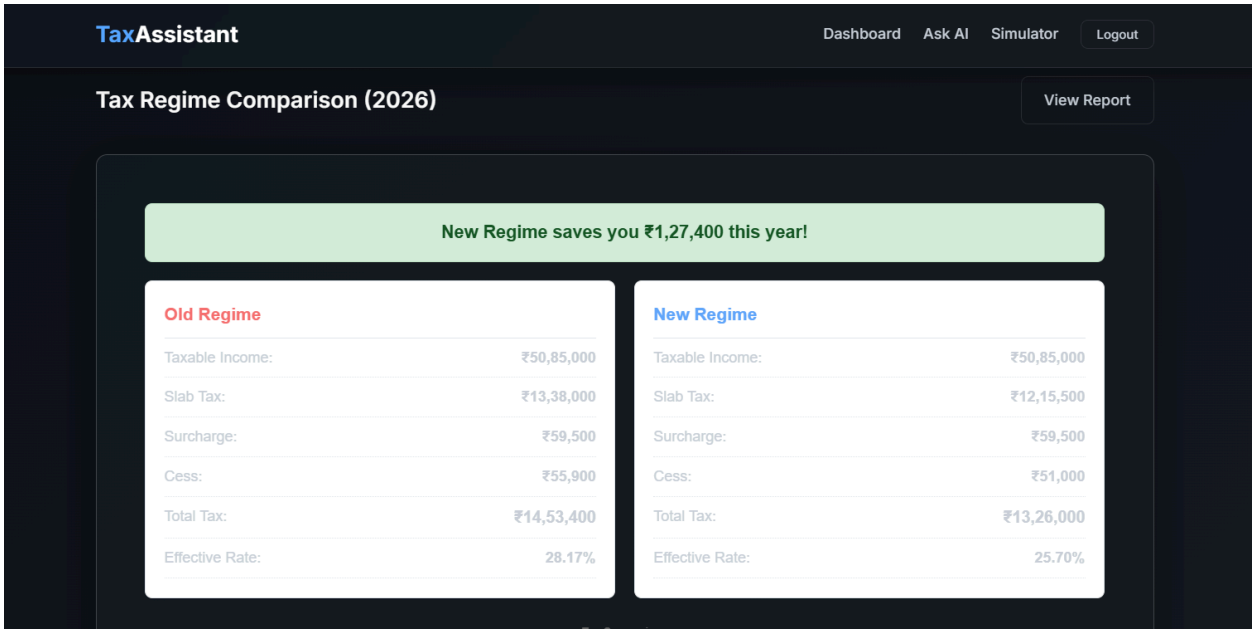
6) **Upload file (salary slip/ section 80A) with pdfParse to automatically fill data**



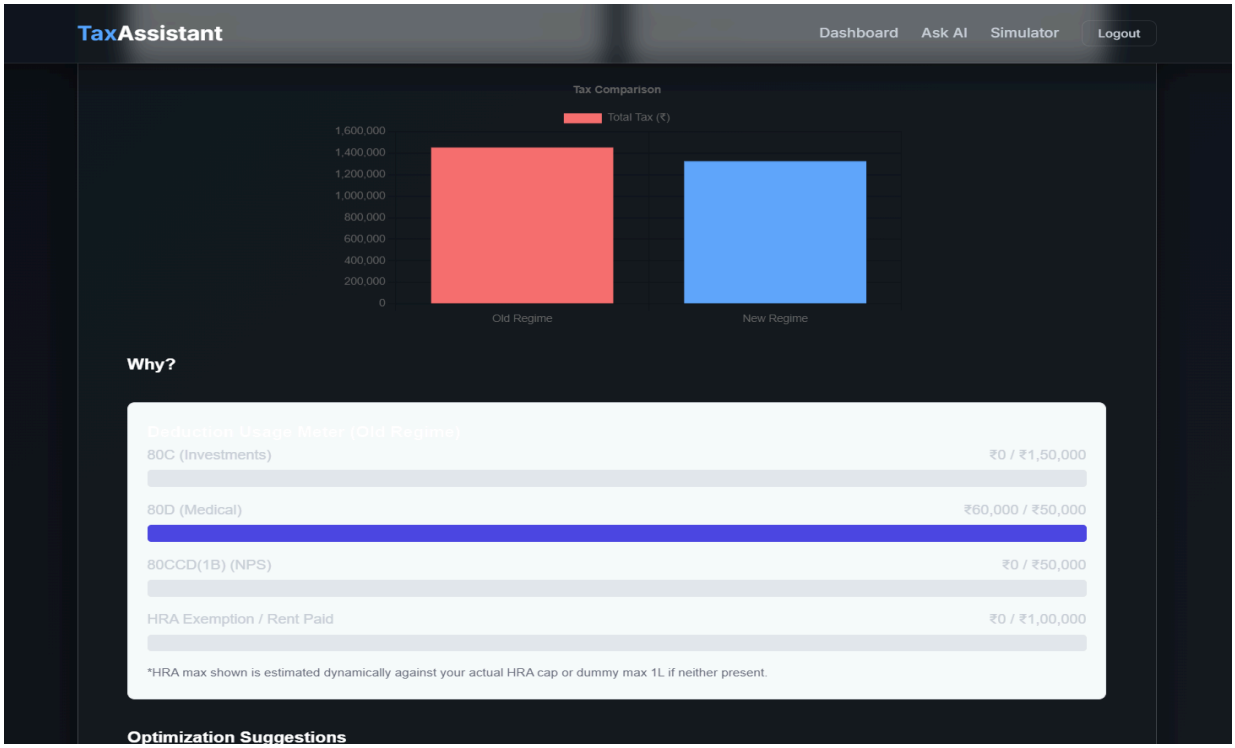
7) **Viewing full Report (tax calculated as per government guidelines)**



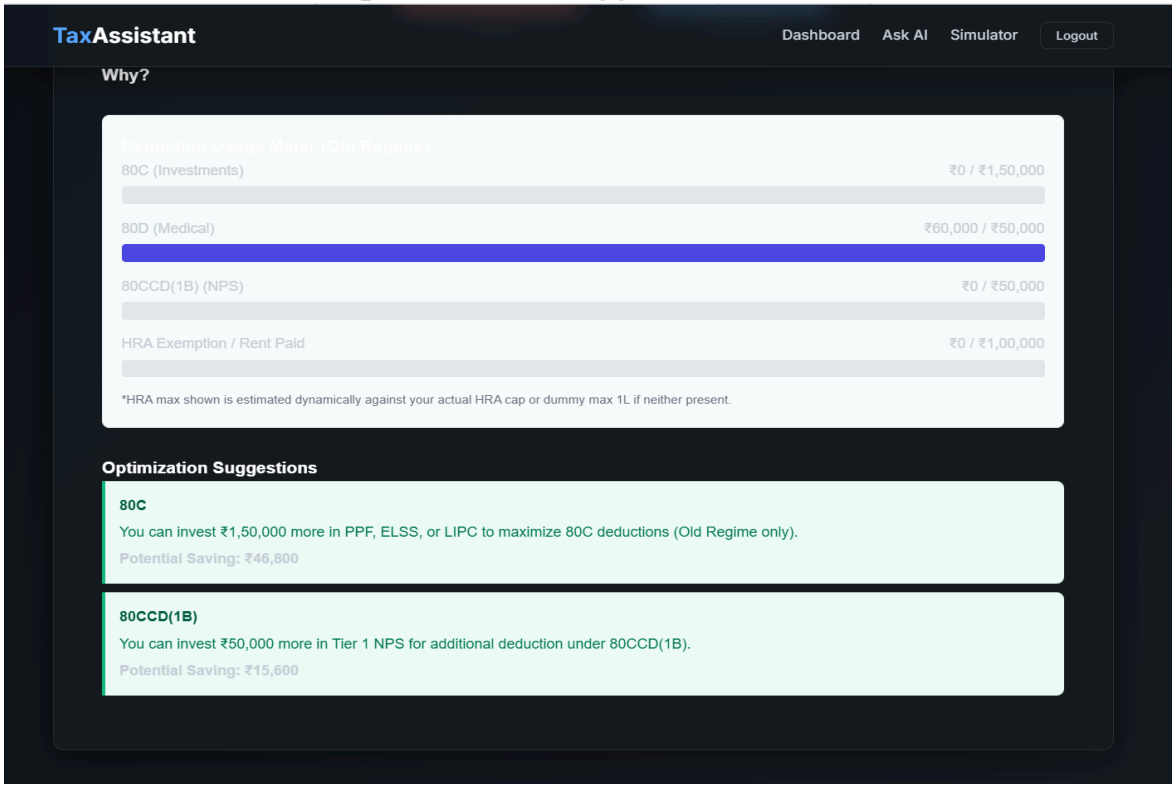
8) Comparing OLD and NEW TAX REGIMES (as per actual Indian Standards)



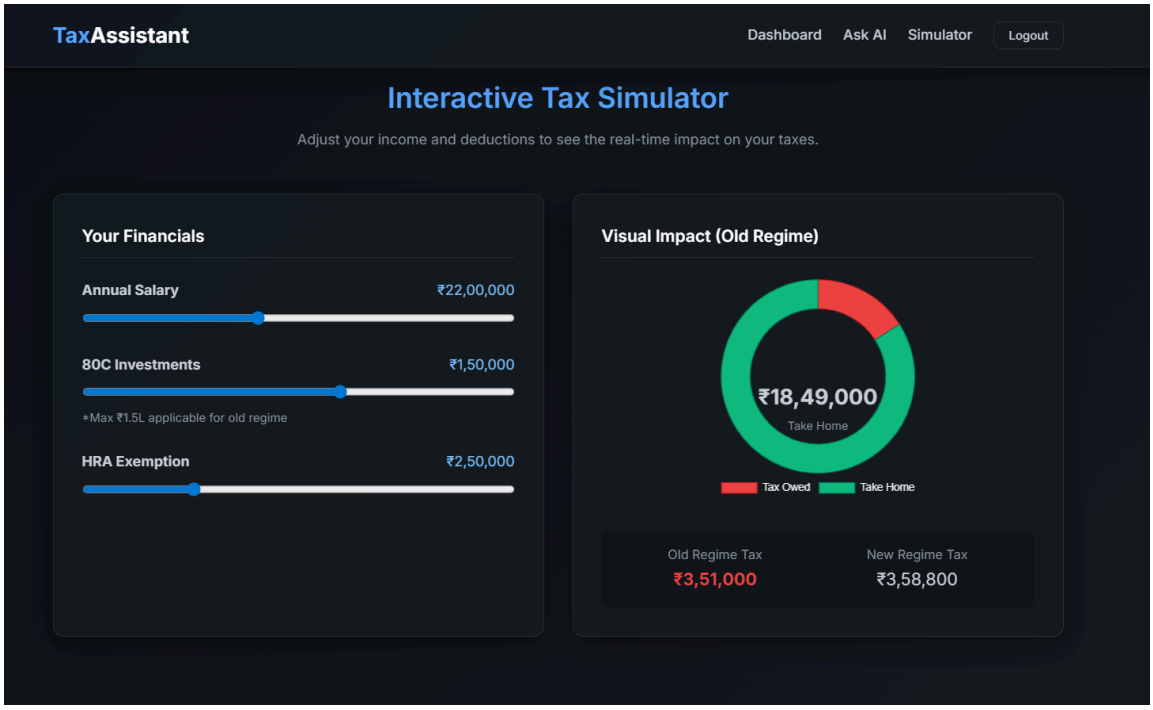
9) Reasoning for the result of the comparison



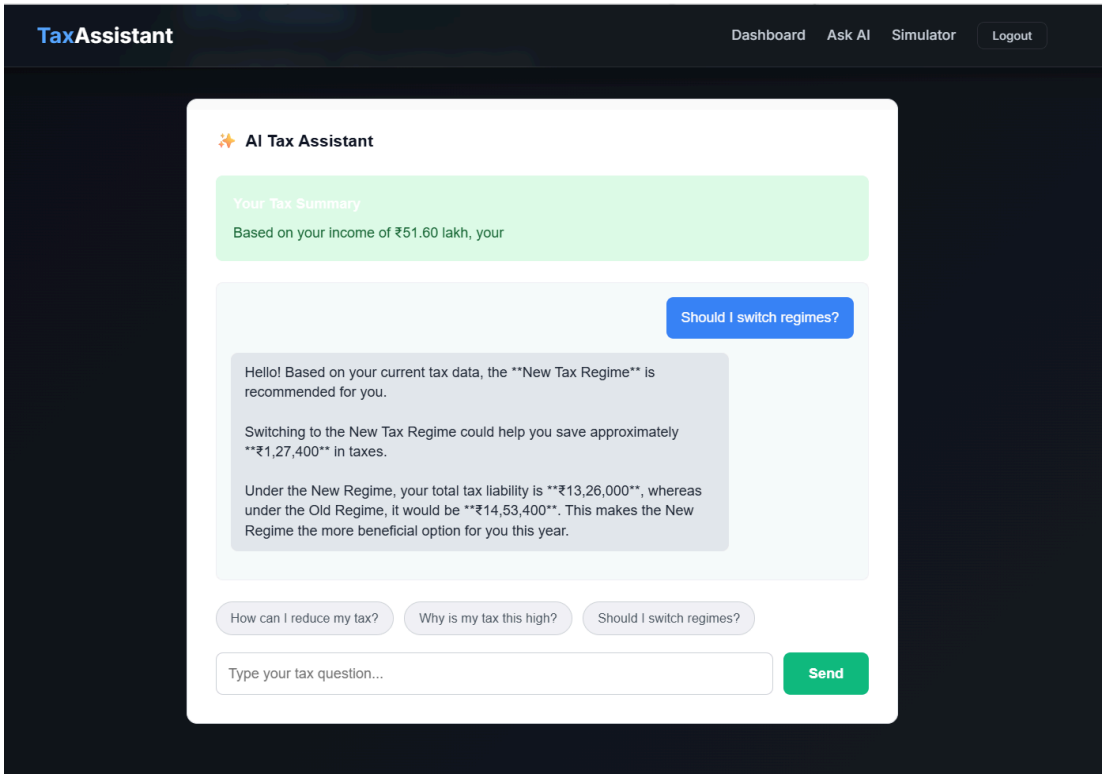
10) Rule-based Optimization Suggestions



11)Tax Simulator with Sliders



12) Context-aware AI assistant to guide the user



Strengths and Limitations

Strengths

- Modular Design: Clear separation of concerns between UI and Logic.
- Developer Experience: Vite provides a significantly faster build pipeline than CRA.
- Security: Implements industry-standard hashing and tokenization.

Limitations & Risks

- Static Rules: Tax brackets are currently hardcoded, making them difficult to update.
- Regional Constraints: Only supports a single tax jurisdiction.
- Performance: Lacks server-side caching (e.g., Redis) for high-traffic scenarios.

Future Enhancements

- Enhanced Security: Implement Refresh Tokens and Rate Limiting to prevent Brute Force attacks.
- Performance: Add database indexing and caching for frequent queries.
- External APIs: Connect directly to banking or government portals for real-time compliance.